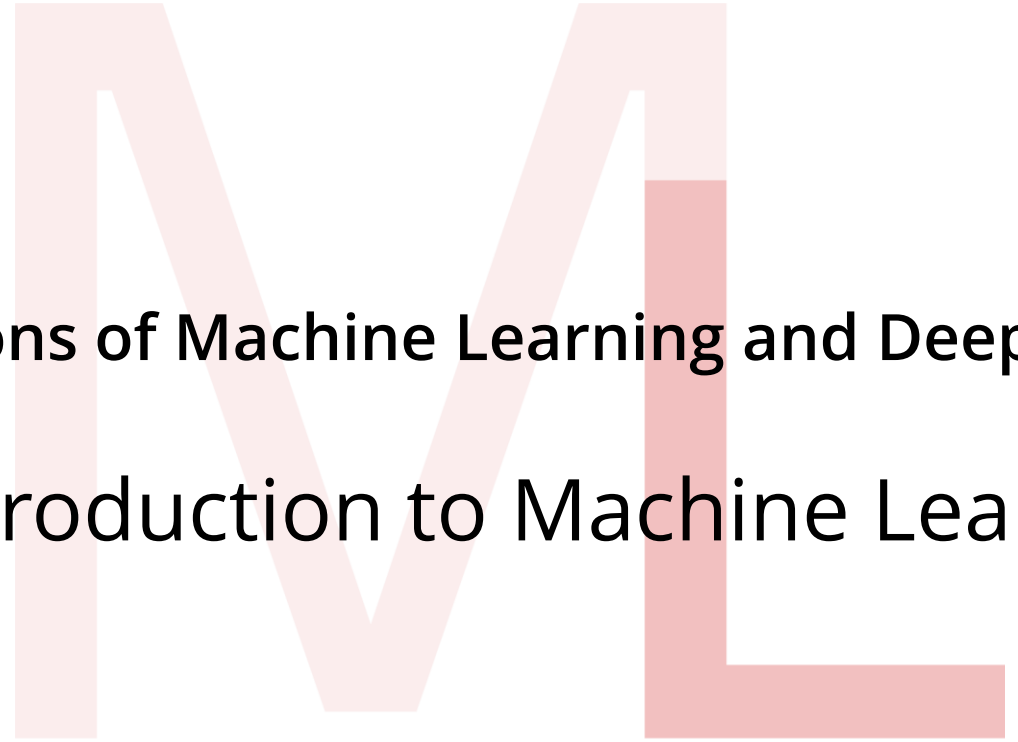
Foundations of Machine Learning and Deep Learning

1. Introduction to Machine Learning



upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Outline

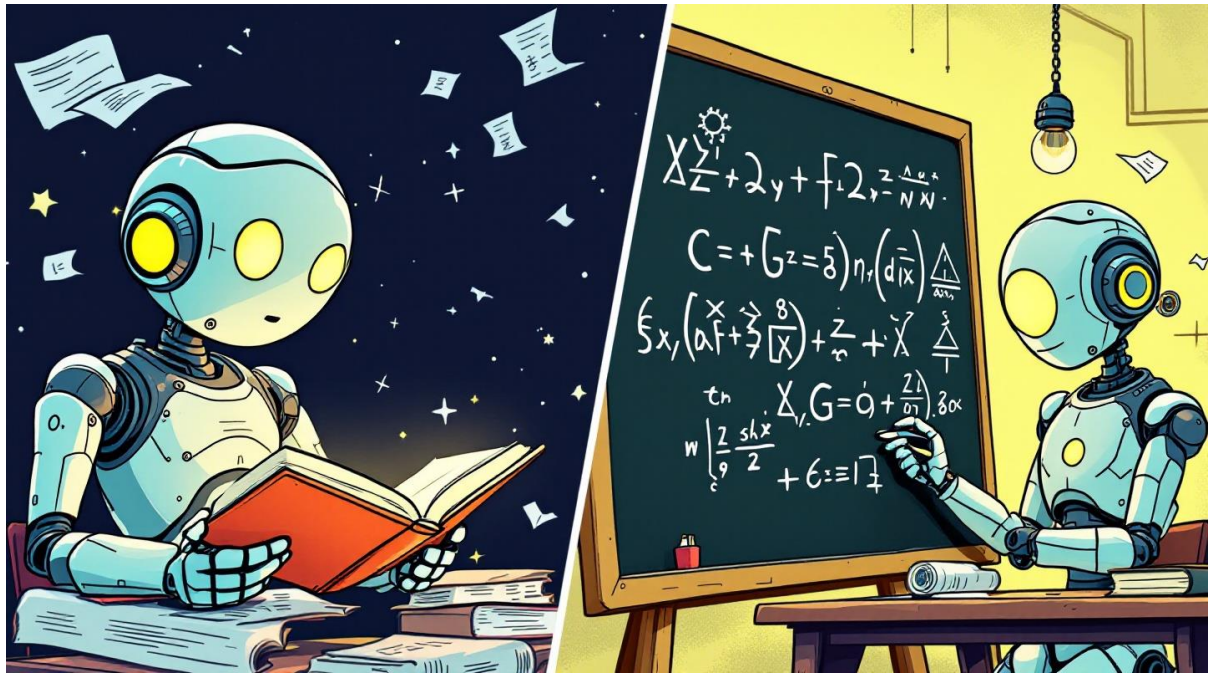
- What is Machine Learning?
- Notation & Core Concepts
- Learning paradigms
- Classical Models
  - K nearest neighbors
  - Naive Bayes
  - Decision trees
  - Ensemble models
- Model evaluation and selection
- Clustering

What is Machine Learning?

What truly makes a system 'learn'?

List 3 or 4 mandatory **conditions** that a software must meet to be considered Machine Learning

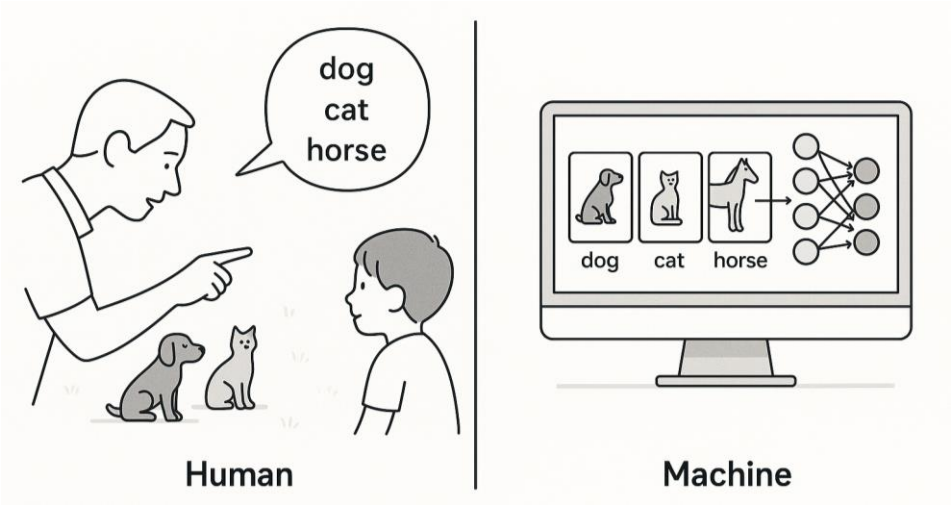
What is Machine Learning?



- The study that focuses on creating **systems capable of performing tasks that normally require human intelligence**, such as learning, reasoning, and problem-solving.

What is Machine Learning?

- Intuitively
  - Machines that **imitate humans**
  - Machines that **are intelligent**
  - Machines that are **capable of learning**
    - **How? From experience (examples)**



What is Machine Learning?

- Which of the following systems are actually using Machine Learning?
  - System A: A medical app that diagnoses flu based on 50 manual rules written by doctors (e.g., IF fever > 38°C AND cough THEN flu).
  - System B: Spotify's algorithm generating a "Discover Weekly" playlist based on your listening history and other users' habits.
  - System C: An e-commerce system that sends a 10€ discount coupon to any customer who hasn't logged in for 30 days.
  - System D: A smart thermostat that automatically turns on the heating when your phone gets within 2 km of your home.
  - System E: A bank's fraud detection system that flags a transaction as suspicious because it looks completely different from all your historical spending patterns.
  - System F: A chess engine like Deep Blue (1997) that evaluates 200 million possible board moves per second using a fixed mathematical scoring function.

What is Machine Learning?

- Machine Learning is about building systems that **recognize patterns** and **make decisions** based on **data**
- A Machine Learning problem has three key elements:
  - The task
    - What are we trying to achieve?
      - Classifying images, detecting fraud, forecasting demand…
  - The data
    - What examples or observations can the model learn from?
      - Labeled datasets, user interactions, sensor readings…
  - The evaluation criterion
    - How do we know if the model is doing well?
      - Accuracy, MSE, business metrics…

What is Machine Learning?

- Machine Learning is not about memorizing the training data
  - It's about performing well on **new, unseen data** (generalization)

A model is successful not because it fits the training data, but because it generalizes to future examples

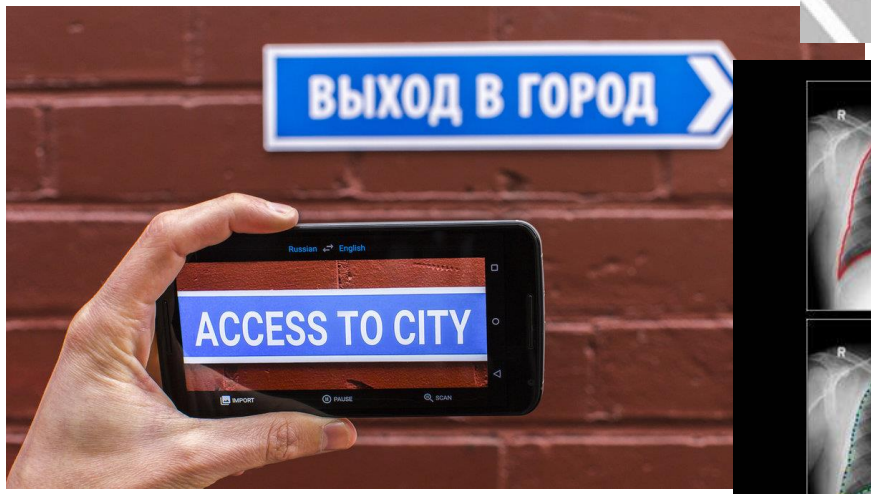


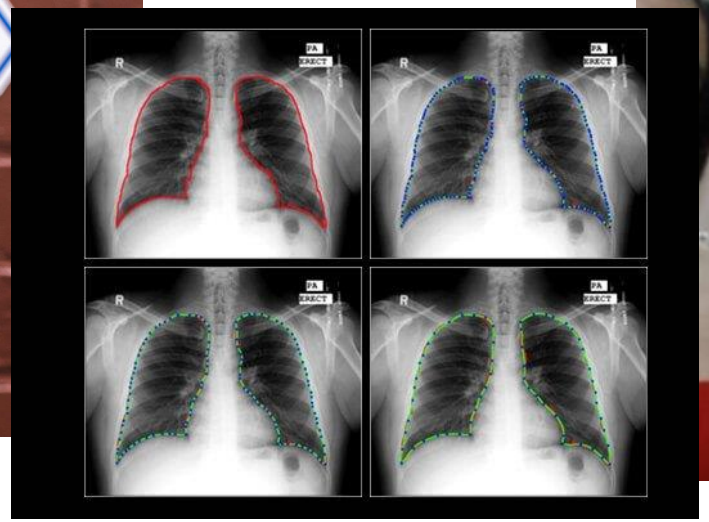
Examples of classical Machine Learning





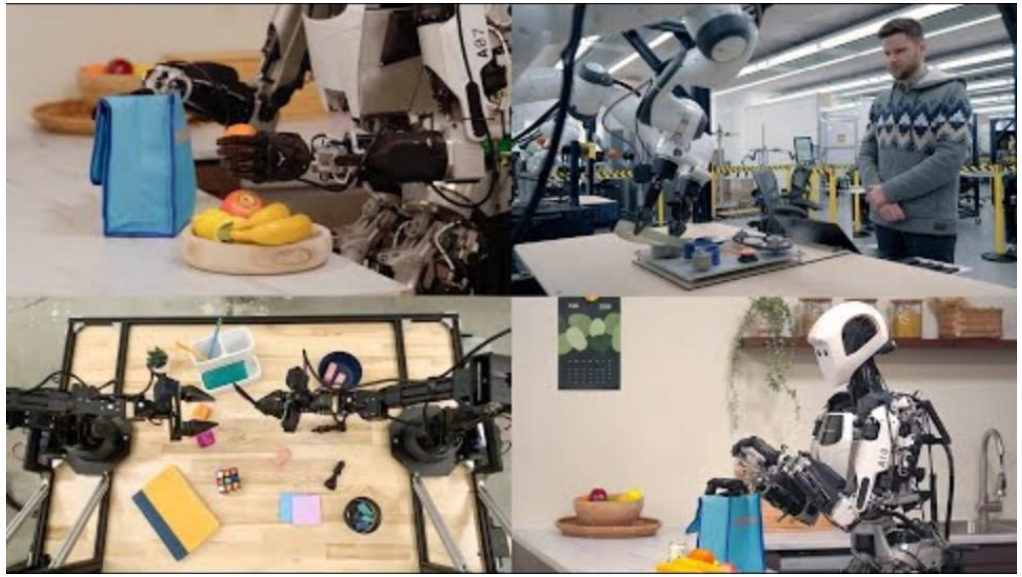








Examples of recent Machine Learning



ChatGPT

Claude

Gemini

Jasper

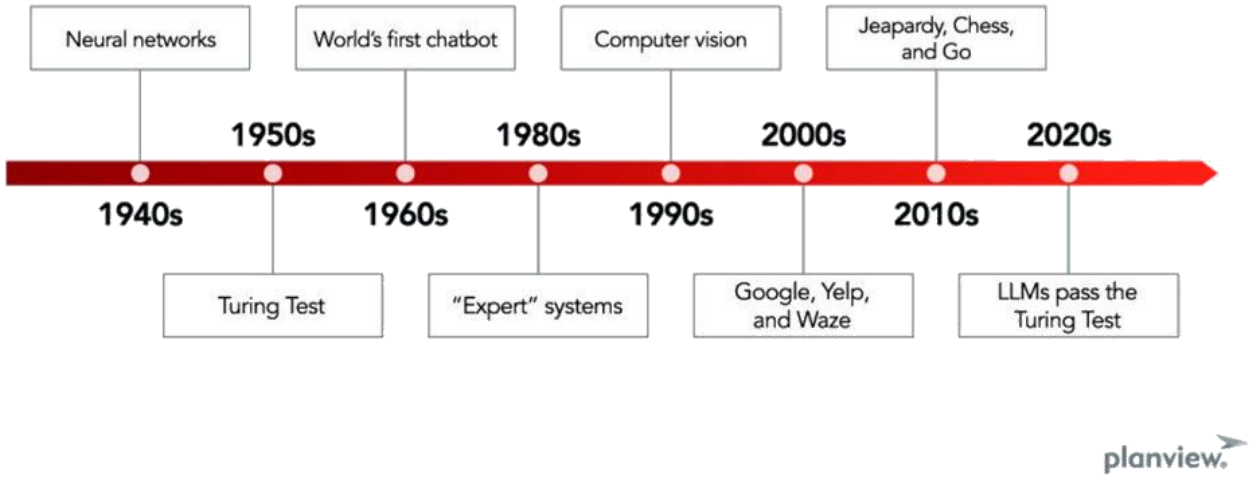
LLAMA 2

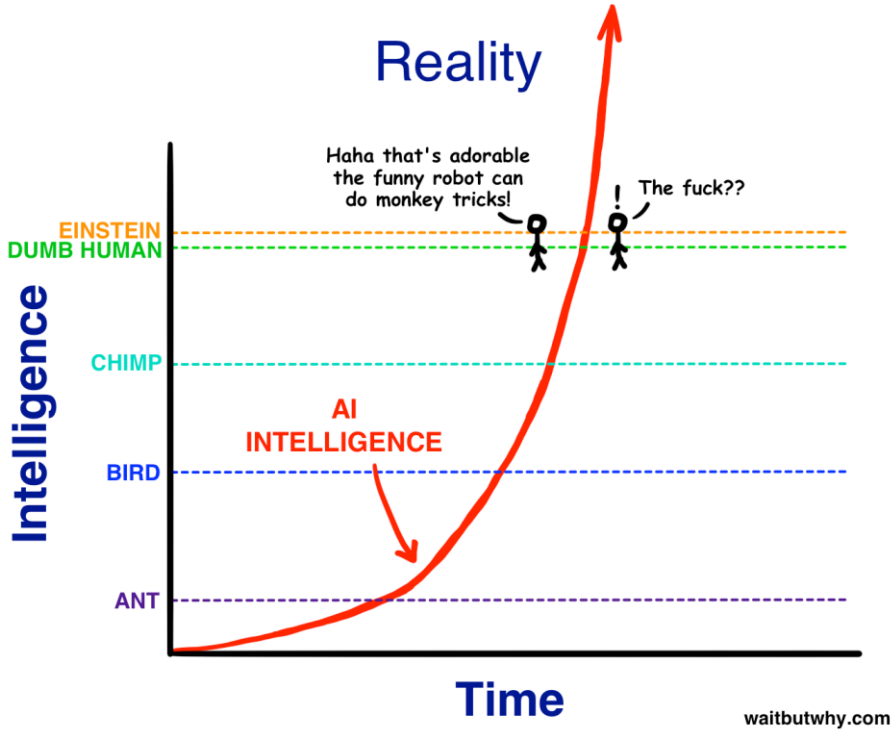






Exponential evolution of Machine Learning





Outline

- What is Machine Learning?
- Notation & Core Concepts
- Learning paradigms
- Classical Models
  - K nearest neighbors
  - Naive Bayes
  - Decision trees
  - Ensemble models
- Model evaluation and selection
- Clustering

Data types and Structures

- What is data?
  - Structured representation of observations / measurement, i.e., a collection of examples (or instances)
  - Each datapoint (instance) is typically an input-output pair:
    - Input: a set of features
    - Output: a label or target (for supervised learning

- Types of data:
  - Tabular data (rows = instances, columns = features)
  - Image data (grids of pixels, possibly channels)
  - Text data (sequences of words/tokens)
  - Sequential, audio, video, graph (each type has its own structure)

Data types and Structures

- From real-world data to ML format
  - Raw data is often incomplete, inconsistent or nested
    - Example: JSON, logs, media…
  - Machine Learning models expect numerical arrays or tensors
  - **Preprocessing is essential to bridge real-world data and model input**

"More data beats a clever algorithm, but better data beats more data."



Andrew Ng
(Stanford, Coursera ML course)

Terminology – Data

- Instance / example / sample / datapoint:
  - One individual data point (e.g., one image, one row in a table)
- Feature / attribute / variable:
  - A measurable property of each instance (e.g., height, word count)
- Feature vector
  - A numerical representation of all features for a single instance ($\boldsymbol{x} \in \mathbb{R}^d$)
- Label / target
  - The ground truth or real-world value (y) we want to learn to predict (e.g., a class, a numerical value)
- Dataset
  - A collection of instances: $\mathcal{D} = \left\{\left(\boldsymbol{x}^{(i)}, y^{(i)}\right)\right\}_{i=1}^{n}$

Terminology – Modeling concepts

- Model
  - A function that maps input features to predicted outputs
  - Usually parameterized: $f(\boldsymbol{x}; \boldsymbol{\theta}) \rightarrow \hat{y}$
- Parameters
  - Learnable components ($\boldsymbol{\theta}$) of the model that are tuned during training (e.g., weights and biases of a neural network)
- Prediction / output
  - The model's estimate of the label for an instance ($\hat{y}$)

Terminology – Learning concepts

- Loss function
  - Measures error for a **single example**
  - Input: one prediction $\hat{y}$, one true label y
    - Example:

$$\ell(\hat{y}, y) = (\hat{y} - y)^2$$

- Objective function
  - Aggregates loss over **many examples**
  - Guides model training by quantifying overall performance
    - Example:

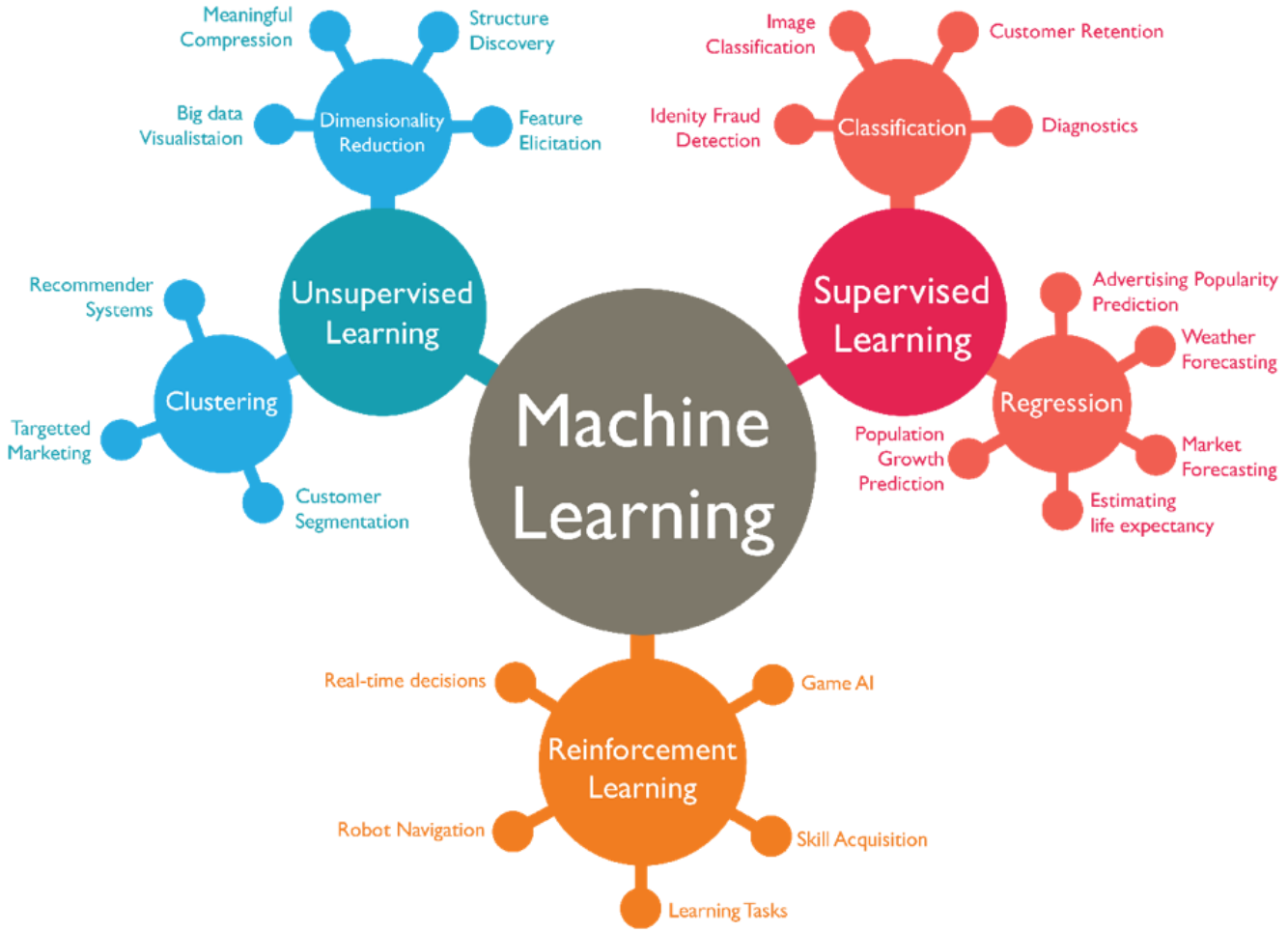
$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell\left(f\left(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right), y^{(i)}\right) = \frac{1}{n}\sum_{i=1}^{n} \ell\left(\hat{y}^{(i)}, y^{(i)}\right)$$

Outline

- What is Machine Learning?
- Notation & Core Concepts
- Learning paradigms
- Classical Models
  - K nearest neighbors
  - Naive Bayes
  - Decision trees
  - Ensemble models
- Model evaluation and selection
- Clustering

Learning paradigms

- **Three main paradigms**:
  - Supervised learning
  - Unsupervised learning
  - Reinforcement learning

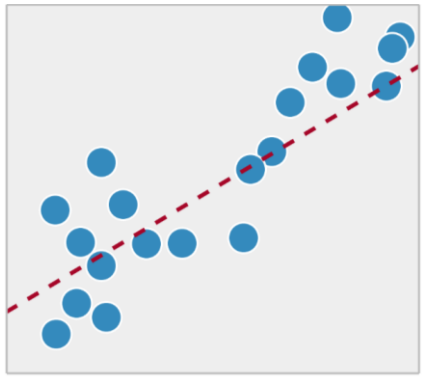


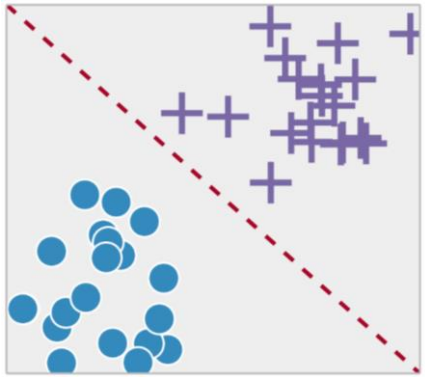
Learning paradigms

Supervised Learning

- Learn from labeled input-output pairs
- The objective is to minimize the prediction error on known targets
- Most widely used paradigm in practice

- Examples:
  - Regression (e.g., price prediction, risk estimation)
  - Classification (e.g., object recognition, sentiment analysis)



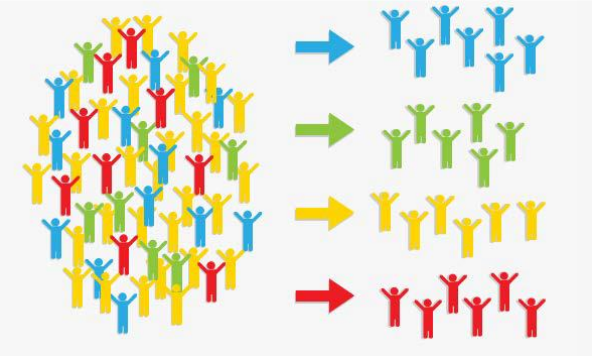


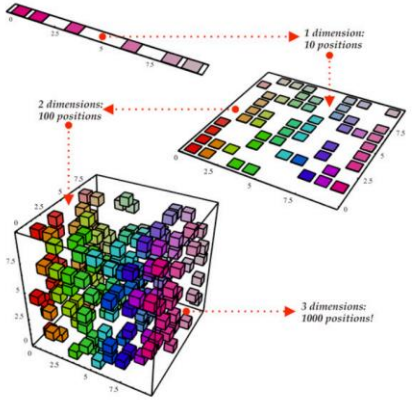
Learning paradigms

Unsupervised Learning

- Discover hidden patterns or structure in unlabeled data
- No explicit output labels are provided
- Useful for exploratory analysis and representation learning

- Examples:
  - Clustering (e.g., customer segmentation)
  - Dimensionality reduction (e.g., visualization, compression)

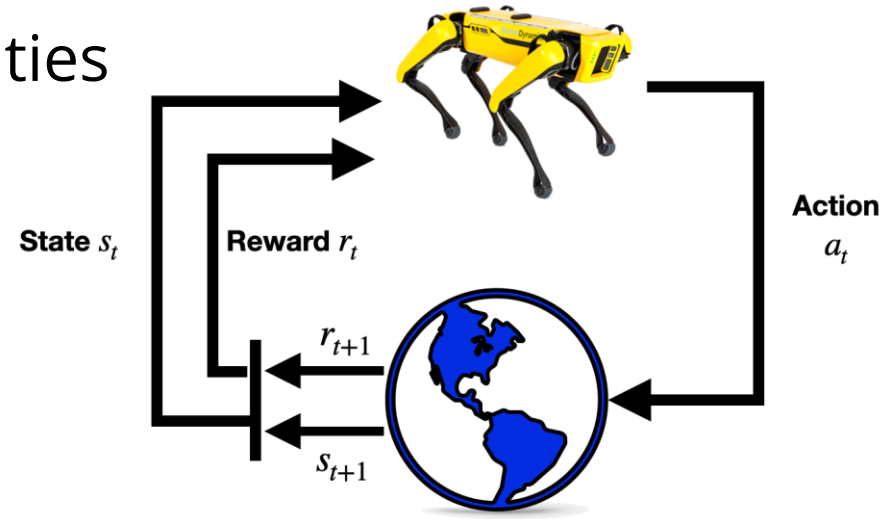




Learning paradigms

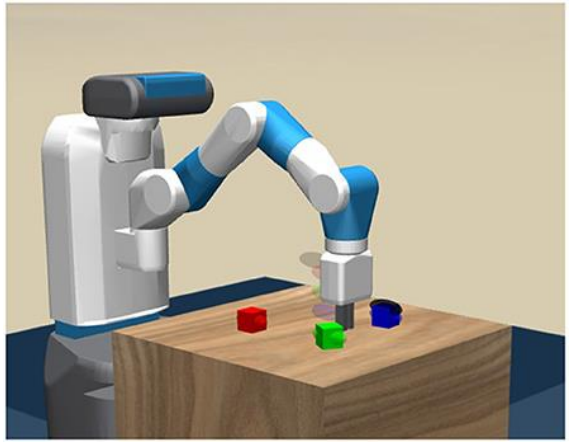
Reinforcement Learning

- Learn by interacting with an environment to maximize long-term reward
- Decision making under uncertainty
- Agent receives feedback via rewards or penalties



- Examples:
  - Game playing (e.g., strategy optimization)
  - Robotics (e.g., control and navigation)

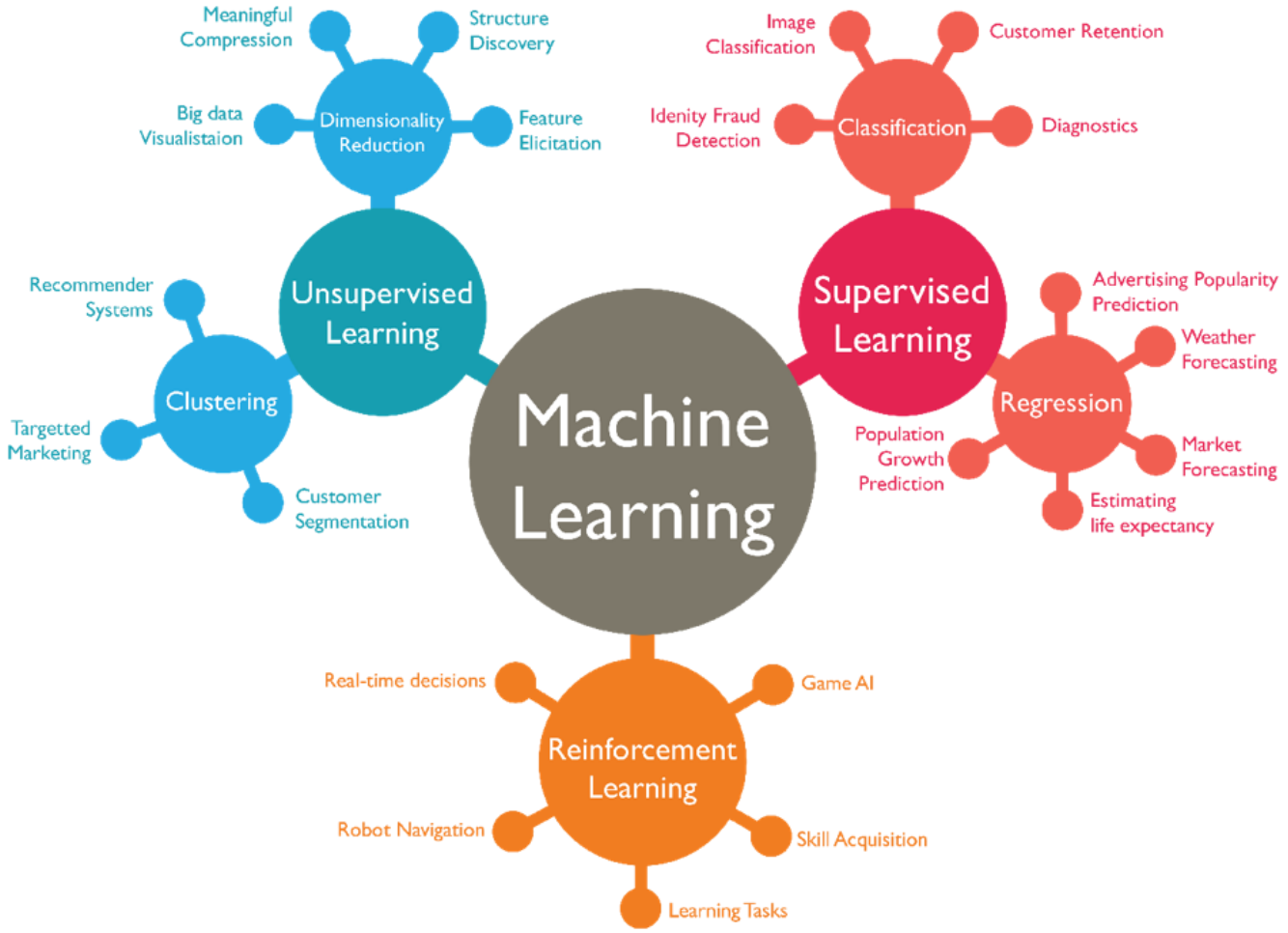




Learning paradigms

- **Three main paradigms**:
  - Supervised learning
  - Unsupervised learning
  - Reinforcement learning

- **Other paradigms**:



Learning paradigms

- Scenario 1: The Expensive Doctor
  - A hospital has 100,000 unlabelled X-ray images. Hiring an expert radiologist to label them (healthy or tumor) costs too much time and money, so only 500 images have official labels. Can we get information from both, labelled and unlabelled, images?

- Scenario 2: The Data-Rich Highway
  - You want to train a computer vision model to detect objects, but you have zero budget to hire humans to draw bounding boxes or label millions of raw images from the internet. How can you extract information from you data to start working?

Learning paradigms

- Scenario 3: The Budgeted Annotator
  - You are building a spam filter and receive 10,000 new emails daily, but your human team only has time to manually inspect and label 50 emails a day. How can you decide which 50 emails should be labelled daily?

- Scenario 4: The Rare Dog Breed
  - You need to classify 5 extremely rare dog breeds, but you only have 30 photographs in total. Training a deep neural network from scratch with 30 images will result in total failure. What can you do?

Learning paradigms

- Scenario 5: The Non-Stop Stream
  - A credit card fraud detection system processes millions of transactions continuously in real time. The incoming data volume is far too massive to store permanently in RAM or disk to retrain the model from scratch every night. How can you incorporate the new information in your model without forgetting the previous data?

- Scenario 6: The Private Keyboard
  - Apple and Google want to improve auto-correct on smartphone keyboards based on real user typing habits. However, privacy laws and user trust strictly prohibit sending private chat messages to central company servers. How can a central model learn from millions of users globally if no personal data is ever allowed to leave any individual's phone?

Learning paradigms

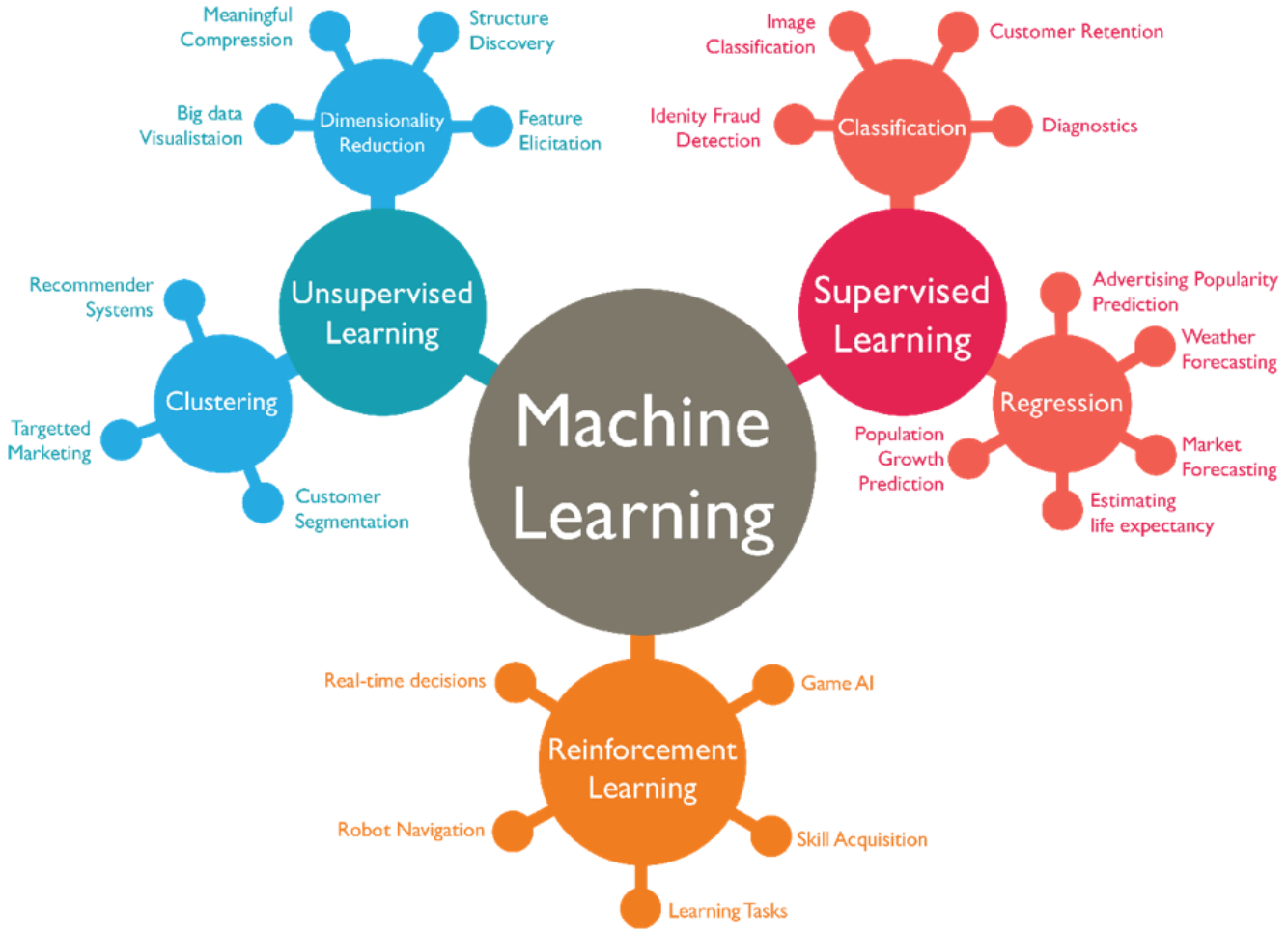
- Scenario 7: The Self-Driving Trilemma
  - An autonomous vehicle needs to perform three tasks simultaneously: detect pedestrians, read speed limit signs, and track lane markings. Is it better to train three separate independent models, or one single shared "backbone" model that outputs three different predictions?

- Scenario 8: Learning How to Learn
  - A medical assistant AI encounters a brand new, undocumented disease and must make accurate predictions after seeing only 2 or 3 patient cases (few-shot learning). If you train a model across thousands of different mini-tasks during development, how can you design it so that it doesn't memorize the tasks, but instead optimizes its ability to adapt rapidly to unseen tasks?

Learning paradigms

- **Three main paradigms**:
  - Supervised learning
  - Unsupervised learning
  - Reinforcement learning

- **Other paradigms**:
  - Semi-supervised learning
  - Self-supervised learning
  - Active learning
  - Transfer learning
  - Online/incremental learning
  - Meta-learning
  - Multi-task learning
  - Federated learning

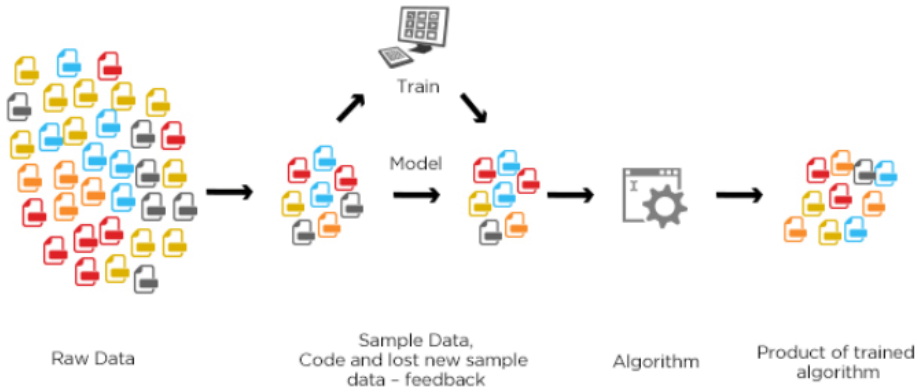


Learning paradigms

Semi-supervised Learning

- Combine labeled data with unlabeled data
- Usually, small amount of labeled and large amount of unlabeled
- Enhances learning when labeling is expensive or time-consuming
- The unlabeled data helps the model generalize better by capturing the underlying data distribution

- Examples:
  - Remote sensing (e.g., satellite image segmentation)
  - Medical imaging (e.g., MRI annotation)
  - Speech recognition (e.g. transcription)



Learning paradigms

Self-supervised Learning

- Learn by predicting parts of the data from other parts
- No external labels – data supervision is internally generated
- Often used for pretext tasks – to learn meaningful representations from unlabeled data
  - The task is not the goal, but a means to learn features that can transfer to other tasks
- Can be seen as a form of unsupervised learning

- Examples:
  - NLP (e.g., masked language modeling)
  - Vision (e.g., predict relative patch position)

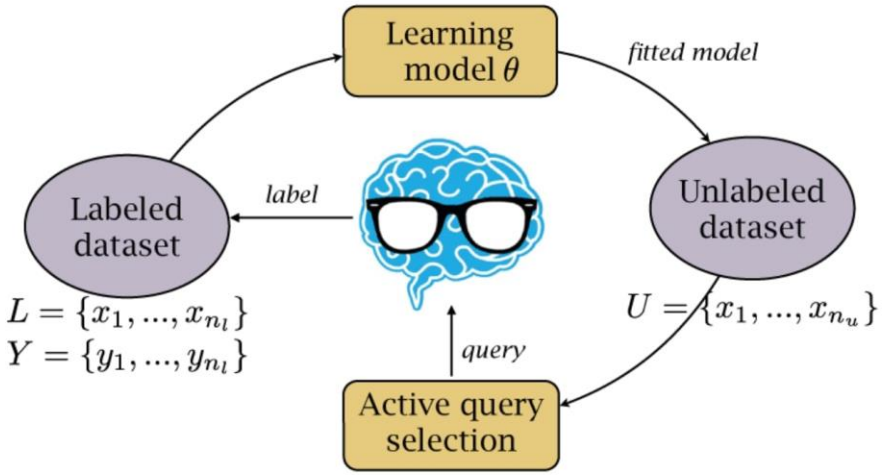


Learning paradigms

Active Learning

- Select the most informative data points to be labeled
- Reduces labeling cost while maintaining model quality
- Requires a querying strategy and human-in-the-loop feedback
  - The model guides the selection of data to label

- Examples:
  - Medical images (e.g., ask radiologists to label only uncertain X-rays)
  - Sentiment analysis (e.g., label edge cases where polarity in unclear)



Learning paradigms

Transfer Learning

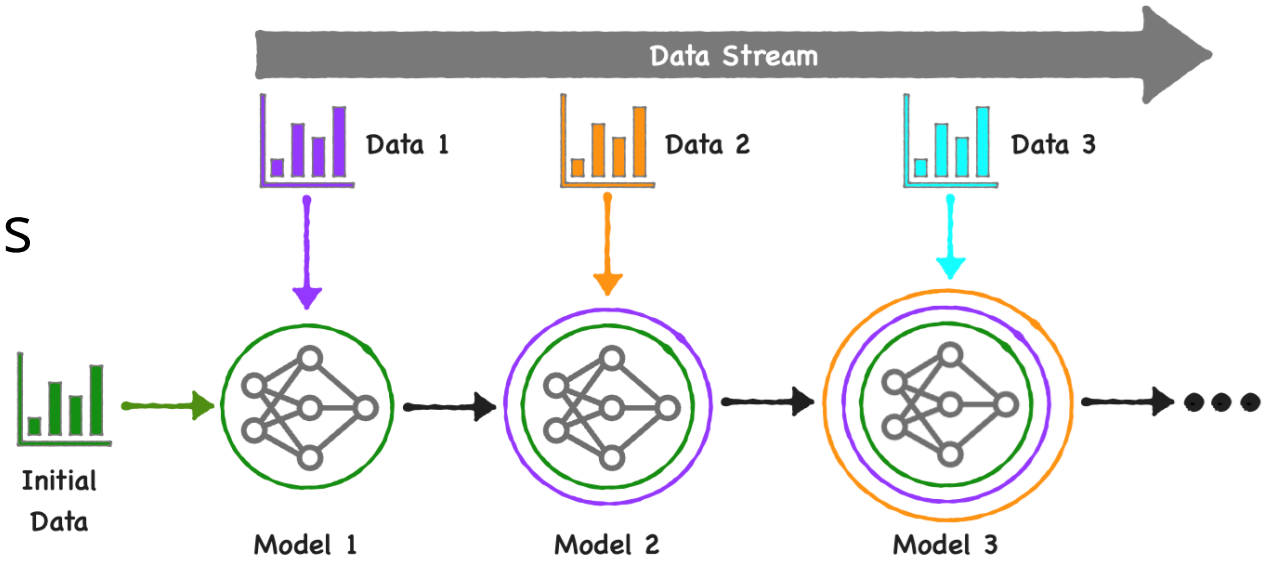
- Transfer knowledge from one task or domain to another
- Reduces the need for large labeled datasets in the target task
- Especially valuable in domains with limited data
- Typical setups:
  - Use pre-trained model to extract features + train new classifier
  - Fine-tuning: continue training on a new dataset
  - Frozen layers + new head: keep pretrained backbone, train new layers
- Examples:
  - Image classification (e.g., ImageNet → specific domain)
  - Cross-domain adaptation (e.g., synthetic to real-world)

Learning paradigms

Online / incremental Learning

- Update the model continuously as new data arrives
- Suitable for streaming data and evolving environments
- Enables models to adapt in real-time
- Common scenarios: incremental class learning, concept drift, etc.

- Examples:
  - Spam filtering
  - Stock price prediction
  - Online recommendation systems

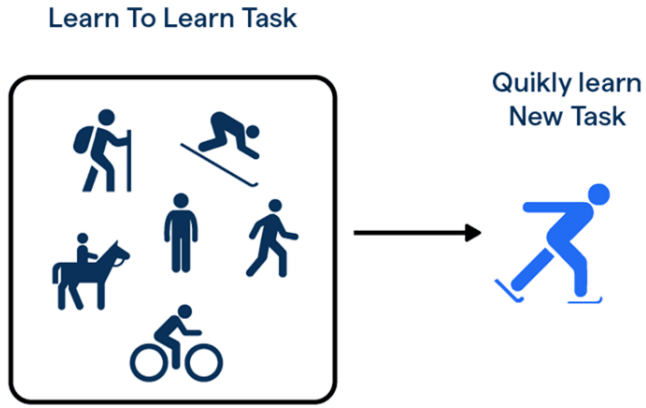


Learning paradigms

Meta-Learning (learning to learn)

- Learn how to quickly adapt to new tasks using prior experience
- Focuses on generalization across tasks, not just within a task
- Common in low-data settings
- Several approaches
  - Learning an initialization that can be fine-tuned quickly
  - Learning an embedding space that makes few-shot classification easy
  - Model that learns how to update itself during task learning

- Examples:
  - Few-shot learning (e.g., recognizing new categories)
  - Task adaptation

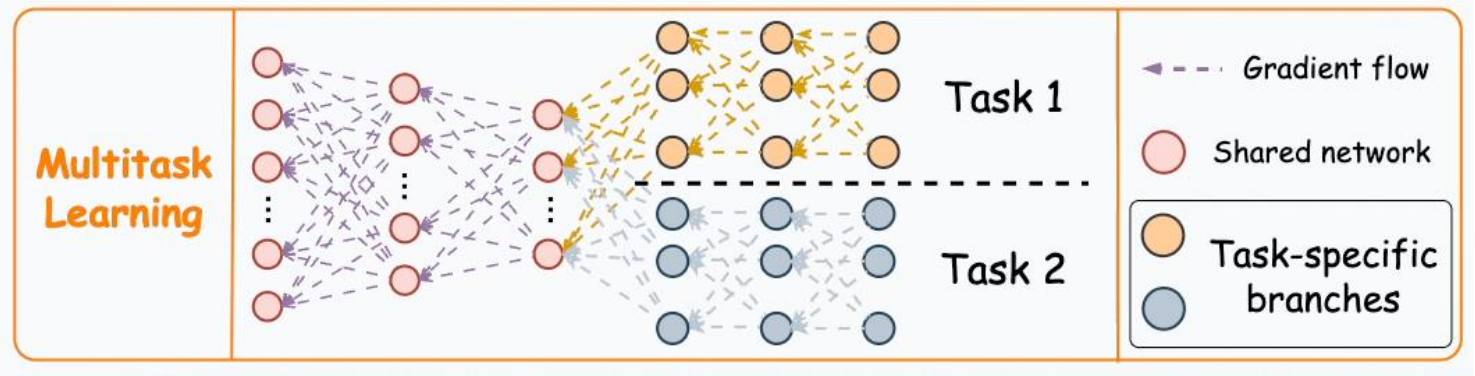


Learning paradigms

Multi-task Learning

- Train a model to perform multiple related tasks simultaneously
- Improves performance through shared learning and regularization
- Helps prevent overfitting on any single task

- Examples:
  - Joint prediction tasks (e.g., object detection + segmentation)
  - Speech (e.g., speaker identification + transcription)

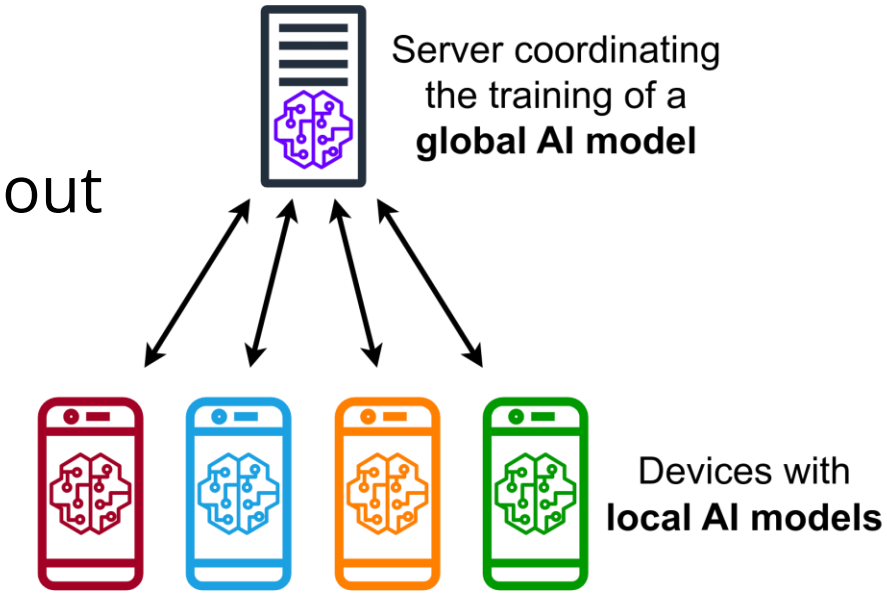


Learning paradigms

Federated Learning

- Learn a model collaboratively across distributed devices without sharing raw data
- Preserves privacy and supports data locality

- Examples:
  - Next word prediction on mobile phones
  - Healthcare (e.g., big diagnostic models without sharing patient data)
  - Smart homes / IoT (e.g., thermostats)

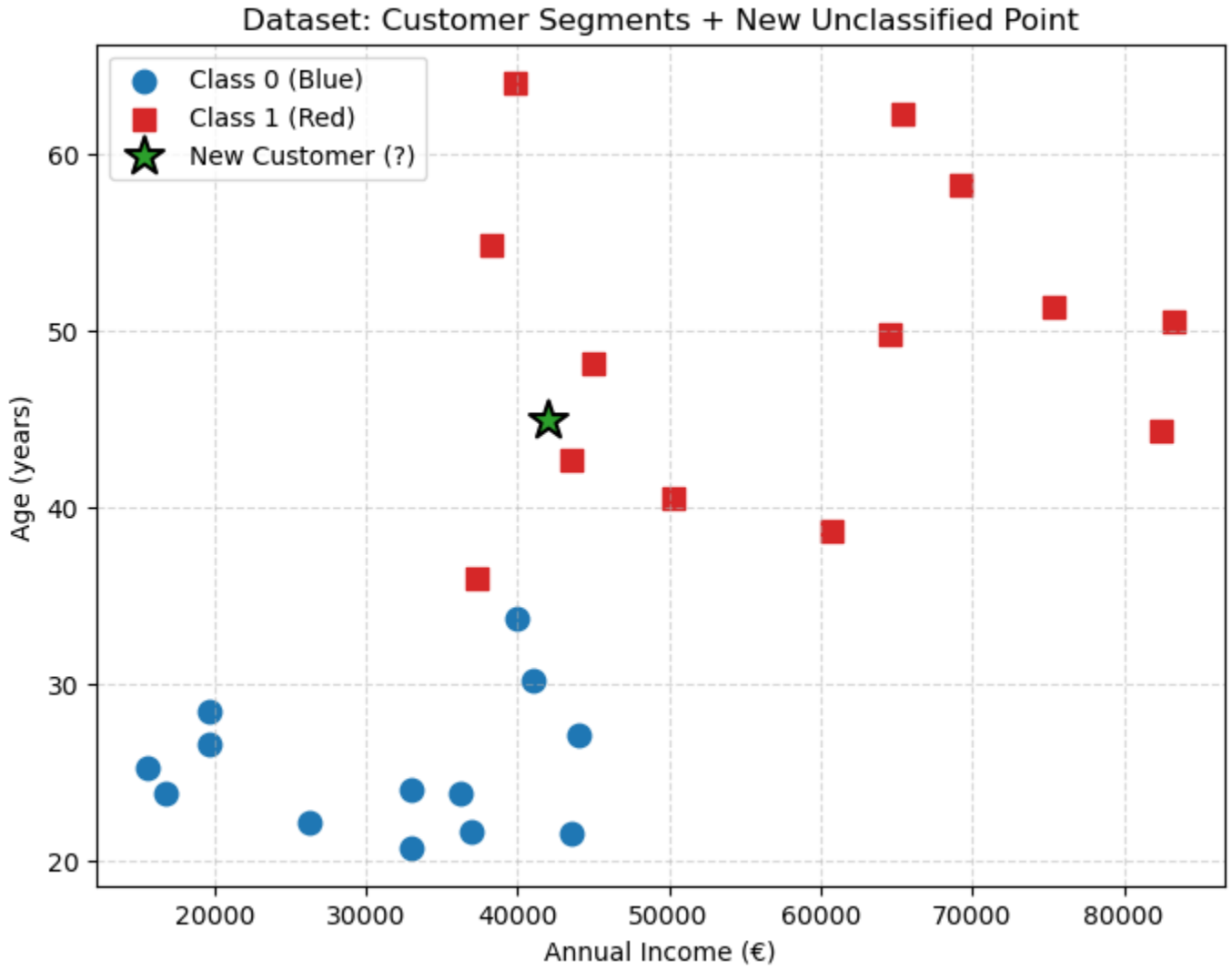


Outline

- What is Machine Learning?
- Notation & Core Concepts
- Learning paradigms
- Classical Models
  - K nearest neighbors
  - Naive Bayes
  - Decision trees
  - Ensemble models
- Model evaluation and selection
- Clustering

Classical Models – kNN

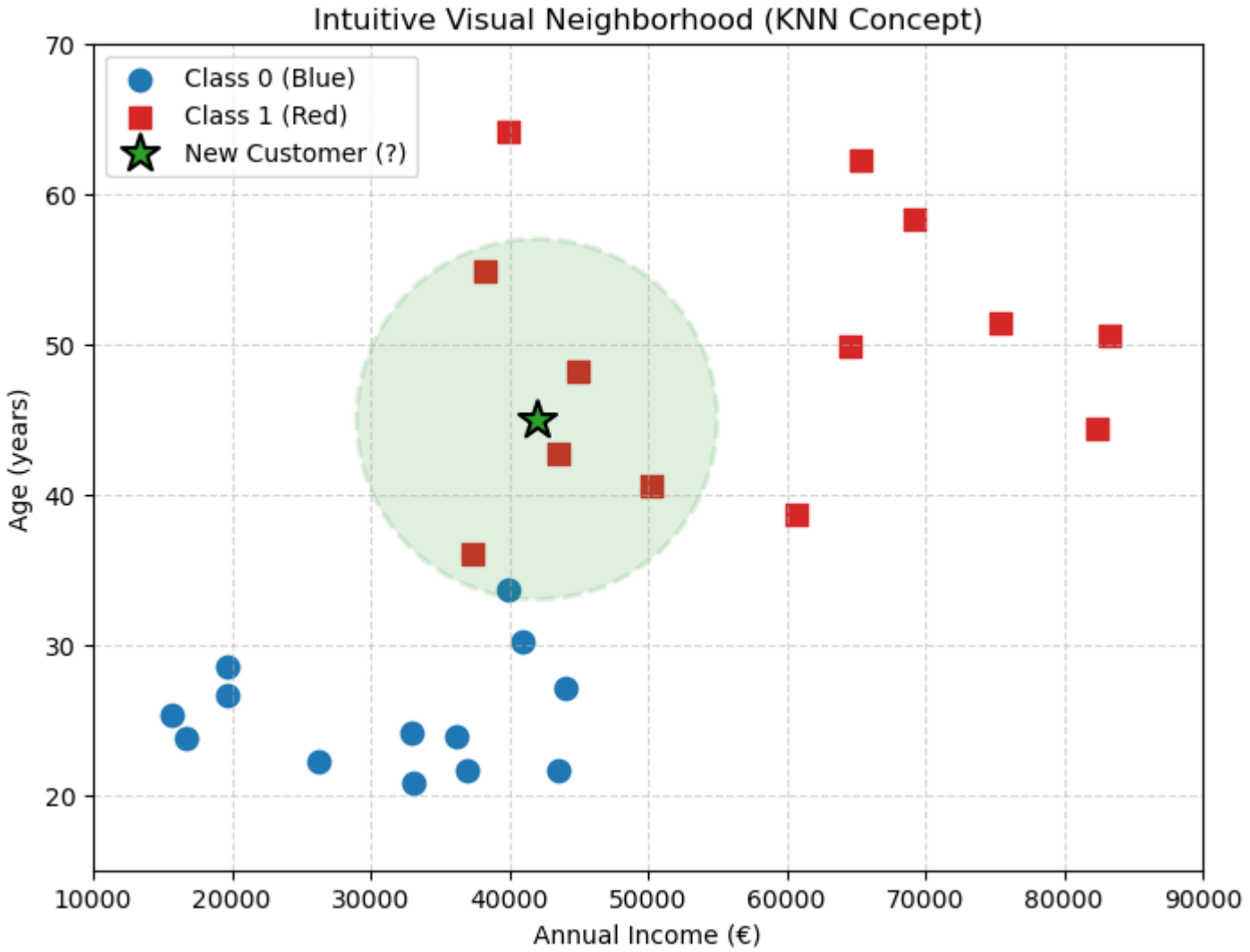
- Which class the new customer belongs to?
- Why?



Classical Models – kNN

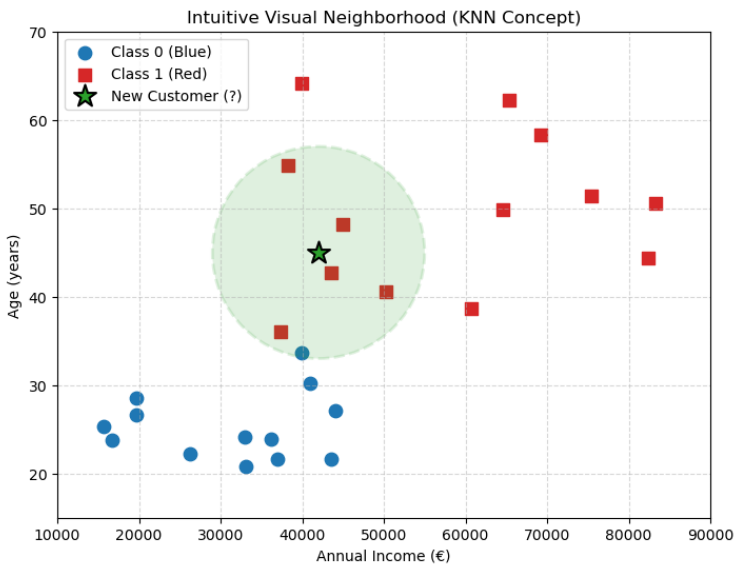
K nearest neighbors (kNN)

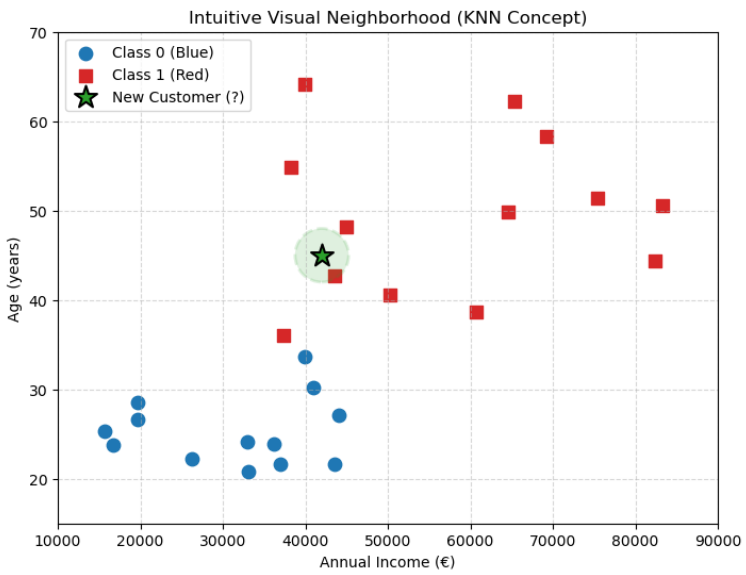
- **Intuition**: Similar inputs → similar outputs
- Predict by looking at the k closest training examples

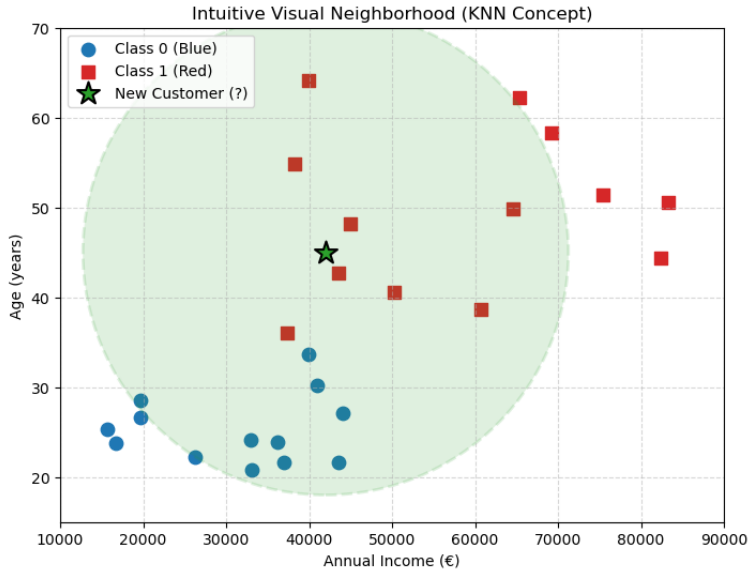


Classical Models – kNN

- Which is the best **k value**?





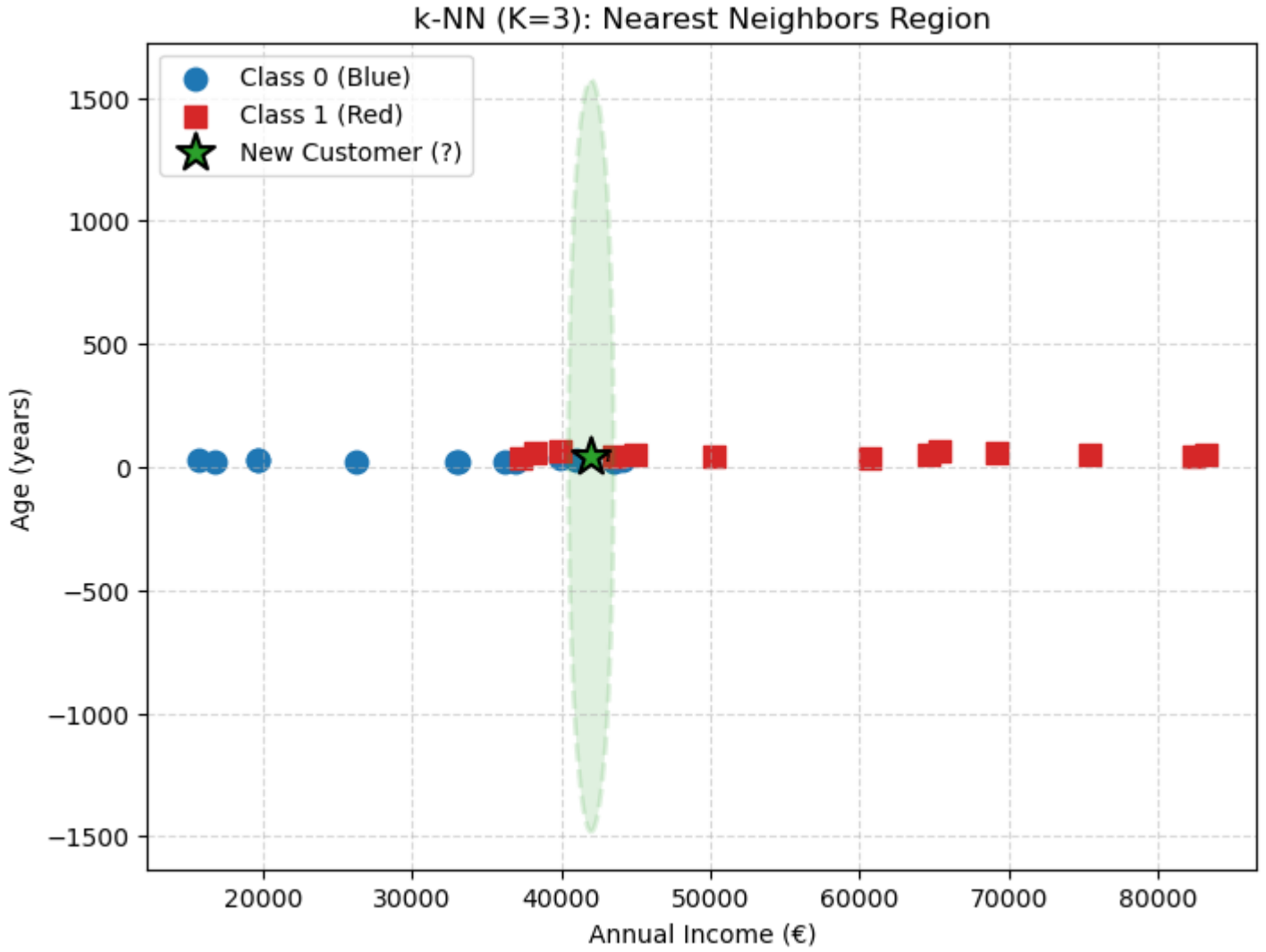


Classical Models – kNN

- Impact of **k value**
  - The chosen value for k affects the performance
    - Low k (e.g., 1)
      - Very flexible
      - Sensitive to noise
      - Might overfit on small local fluctuations
    - High k (e.g., 15):
      - Smooth decision boundary
      - Might misclassify if neighborhood is mixed

Choosing *k* is a trade-off between overfitting and underfitting

Classical Models – kNN



Classical Models – kNN

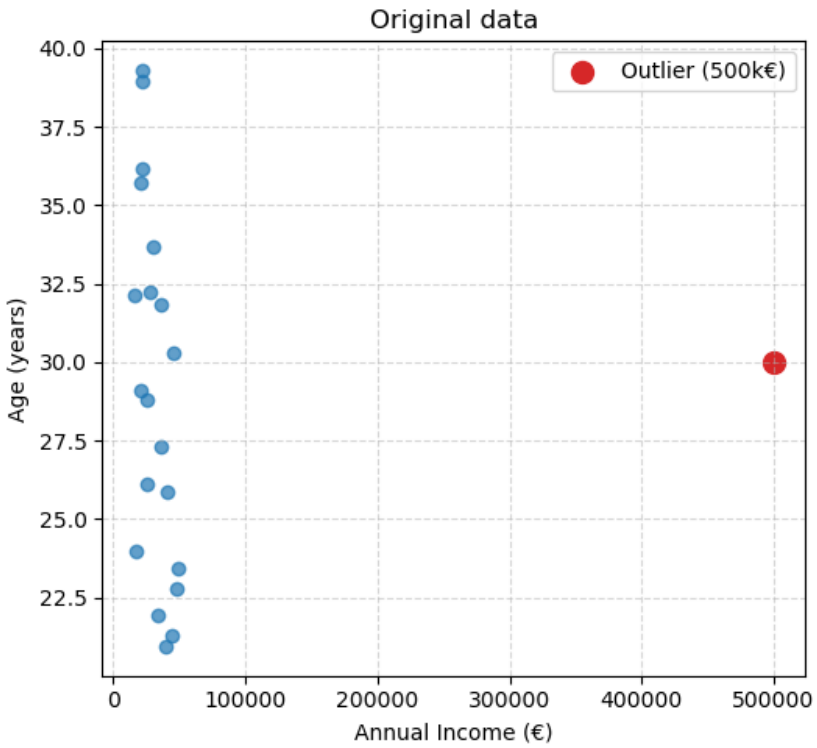
- **Problem**: **Features with larger ranges dominate**
  - A single feature with a wide range (e.g., income 0 – 1000000) can overshadow others (e.g., age 0 – 100)
- **Solution**: normalize or standarize features
  - Min-max normalization:

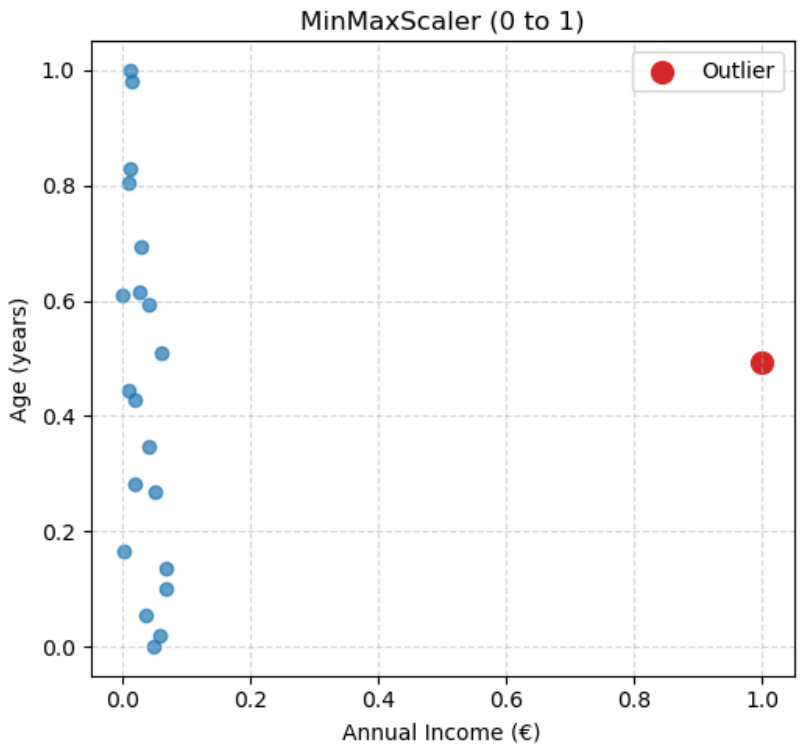
$$x_j^{\text{scaled}} = \frac{x_j - \min(x_j)}{\max(x_j) - \min(x_j)}$$

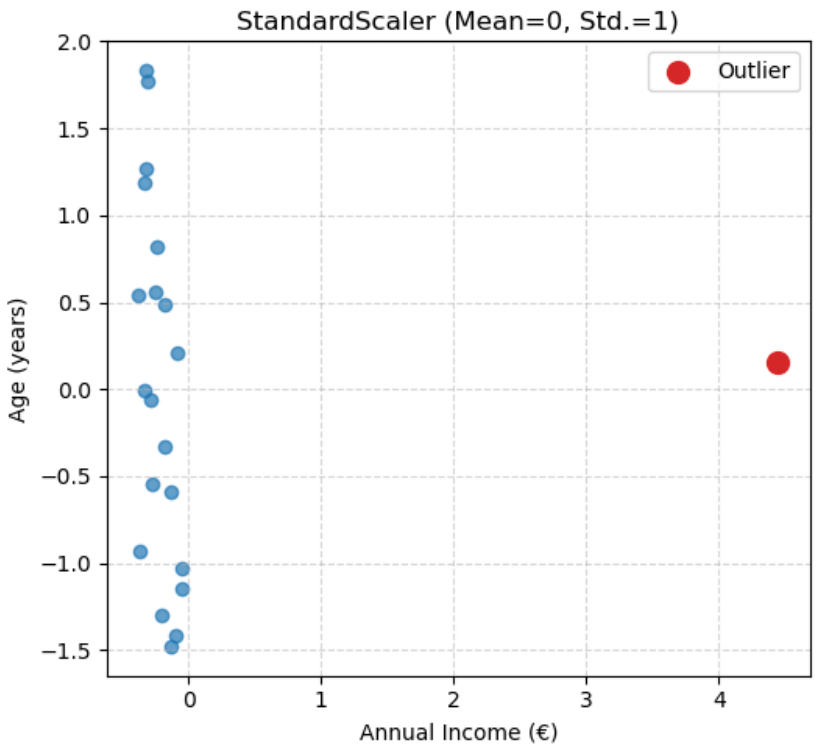
  - Standardization:

$$x_j^{\text{scaled}} = \frac{x_j - \mu_j}{\sigma_j}$$

Classical Models – kNN







Classical Models – kNN

Common distances:

- Euclidean distance

$$\text{dist}_{\text{Euclidean}}(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^{d} (x_j - x'_j)^2}$$

- Manhattan distance

$$\text{dist}_{\text{Manhattan}}(\mathbf{x}, \mathbf{x}') = \sum_{j=1}^{d} |x_j - x'_j|$$

- L_p distance

$$\text{dist}_{L^p}(\mathbf{x}, \mathbf{x}') = \left(\sum_{j=1}^{d} |x_j - x'_j|^p \right)^{1/p}$$

- Cosine distance

$$\text{dist}_{\text{cosine}}(\mathbf{x}, \mathbf{x}') = 1 - \frac{\mathbf{x} \cdot \mathbf{x}'}{\|\mathbf{x}\| \, \|\mathbf{x}'\|}$$

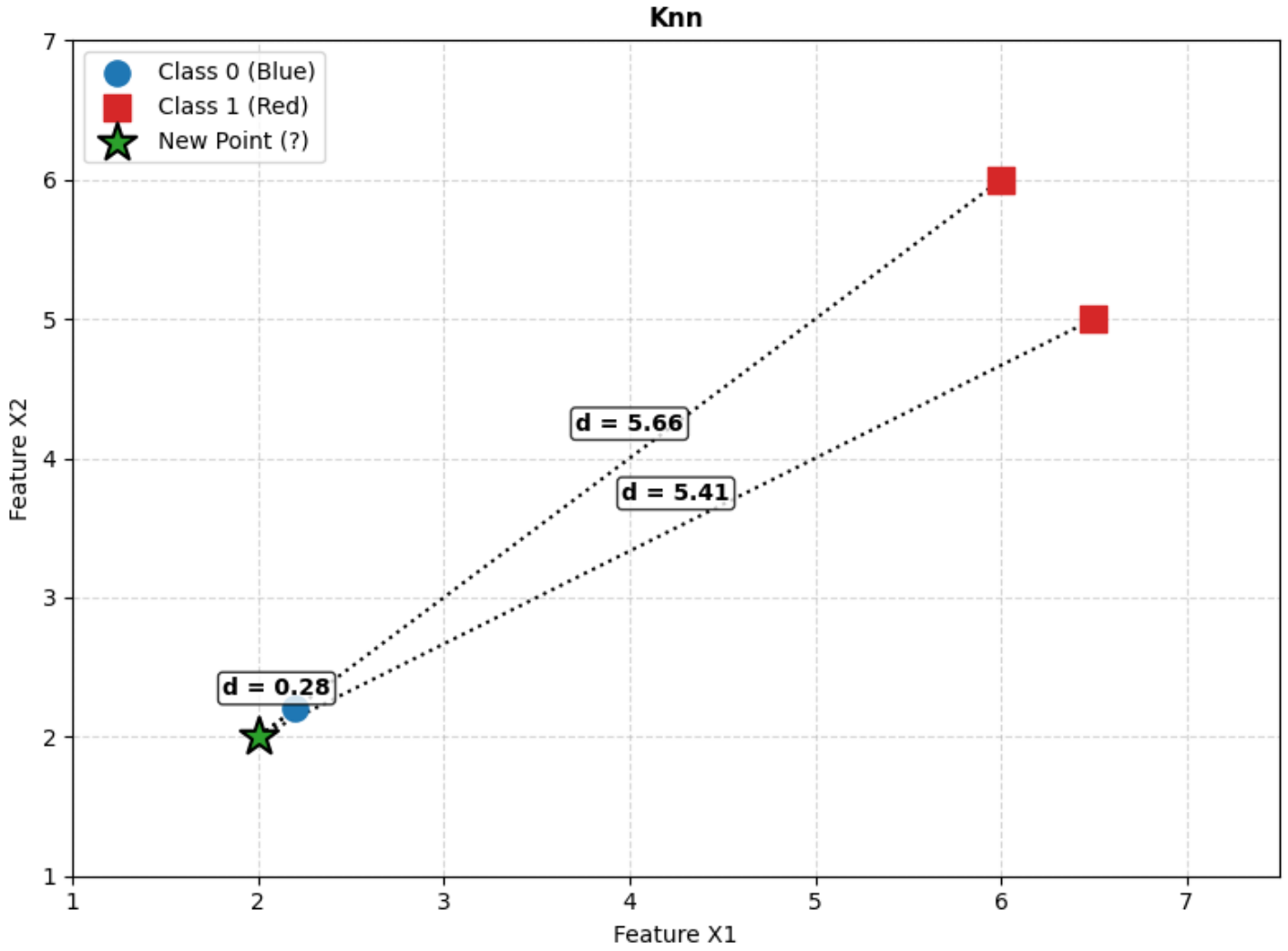
- Mahalanobis distance

$$\text{dist}_{\text{Mahalanobis}}(\mathbf{x}, \mathbf{x}') = \sqrt{(\mathbf{x} - \mathbf{x}')^\top \mathbf{S}^{-1} (\mathbf{x} - \mathbf{x}')}$$

Classical Models – kNN

- **Given:**
  - A labeled training dataset: $\mathcal{D} = \left\{\left(\boldsymbol{x}^{(i)}, y^{(i)}\right)\right\}_{i=1}^{n}$
  - A query point (test instance): $\boldsymbol{x}_{\boldsymbol{query}}$
  - A distance function: (e.g., Euclidean)
  - A value for *k*
- **Steps:**
  - Compute the distance from $\boldsymbol{x}_{\boldsymbol{query}}$ to every point in the training set
  - Select the *k* nearest neighbors
  - Aggregate the neighbors' labels:
    - By the majority vote

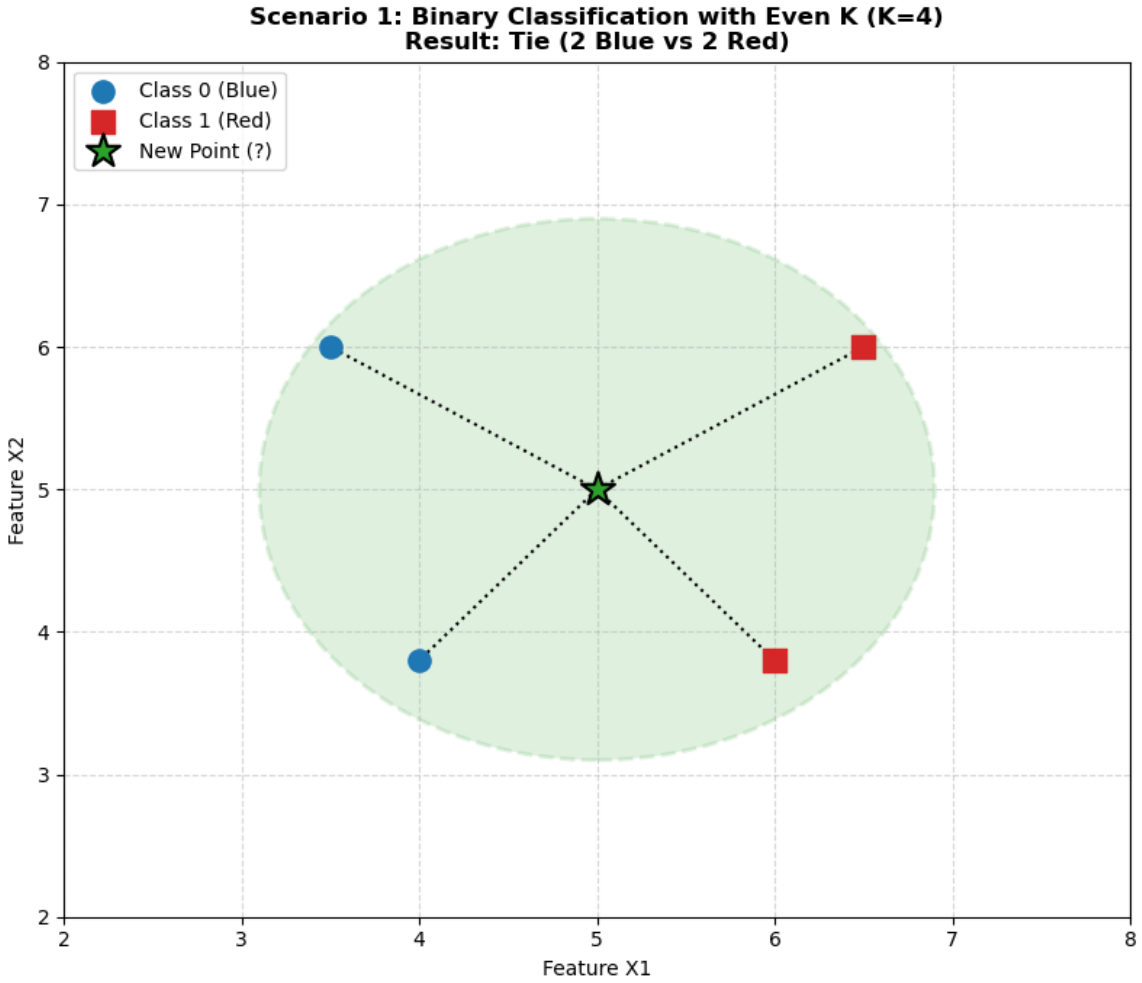
Classical Models – kNN

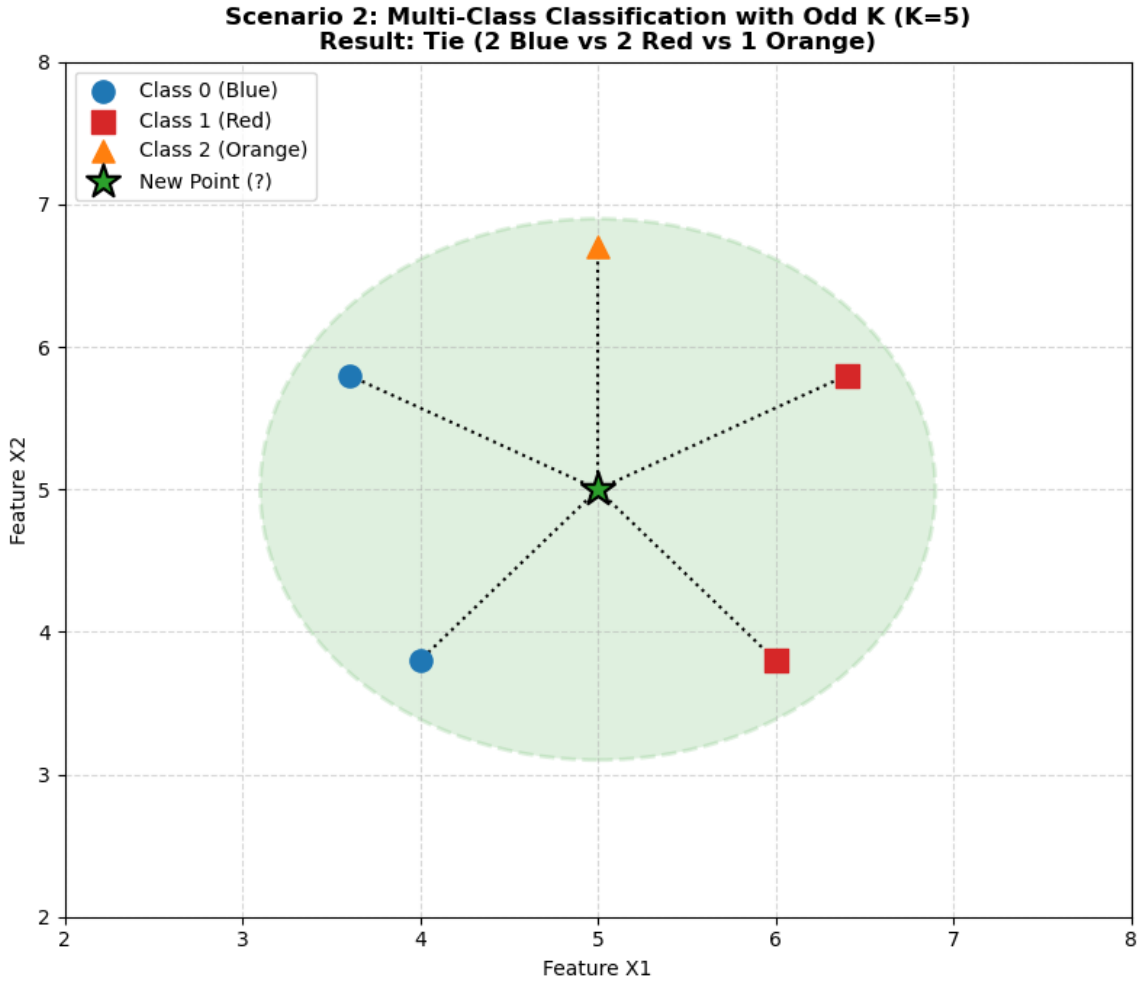


Classical Models – kNN

- **Given:**
  - A labeled training dataset: $\mathcal{D} = \left\{\left(\boldsymbol{x}^{(i)}, y^{(i)}\right)\right\}_{i=1}^{n}$
  - A query point (test instance): $\boldsymbol{x_{query}}$
  - A distance function: (e.g., Euclidean)
  - A value for *k*
- **Steps:**
  - Compute the distance from $\boldsymbol{x_{query}}$ to every point in the training set
  - Select the *k* nearest neighbors
  - Aggregate the neighbors' labels:
    - By the majority vote
      - Alternative: by weighted versions
        - weight neighbors by inverse distance

Classical Models – kNN





Classical Models – kNN

- **Given:**
  - A labeled training dataset: $\mathcal{D} = \left\{\left(\boldsymbol{x}^{(i)}, y^{(i)}\right)\right\}_{i=1}^{n}$
  - A query point (test instance): $\boldsymbol{x}_{\boldsymbol{query}}$
  - A distance function: (e.g., Euclidean)
  - A value for *k*
- **Steps:**
  - Compute the distance from $\boldsymbol{x}_{\boldsymbol{query}}$ to every point in the training set
  - Select the *k* nearest neighbors
  - Aggregate the neighbors' labels:
    - By the majority vote
      - Alternative: by weighted versions
        - weight neighbors by inverse distance

Ties?
- Decide at random
- By closest point
- Majority class
- …

Classical Models – kNN

kNN for regression

- Same idea as classification, but instead of a majority vote, compute a mean
- Predict the average of the target values of the k neighbors

- **Steps:**
  - Compute the distance from $\boldsymbol{x_{query}}$ to every point in the training set
  - Select the *k* nearest neighbors
  - Aggregate the neighbors' labels: $\hat{y} = \frac{1}{k} \sum_{i=1}^{k} y^{(i)}$
    - Alternative: weight the values by the inverse of the distance

$$\hat{y} = \frac{\sum_{i=1}^{k} w_i y_i}{\sum_{i=1}^{k} w_i}, \quad w_i = \frac{1}{\text{dist}(x, x_i) + \epsilon}$$

Classical Models – kNN

Pros and cons

- Pros:
  - Simple, no training required
  - Works well with low-dimensional, clean data
  - Good baseline

- Cons:
  - Slow inference time (search every time)
  - Sensitive to irrelevant features
  - Struggles in high-dimensional spaces (curse of dimensionality)

Classical Models – kNN

- **Curse of dimensionality:**
  - As the dimension increases, distances become less meaningful
  - In low dimensions (2D, 3D), nearest neighbors are “close”
  - In high dimensions, data becomes sparse, and:
    - Every point is far from every other point
    - Neighborhoods lose meaning
    - kNN has trouble finding truly similar points

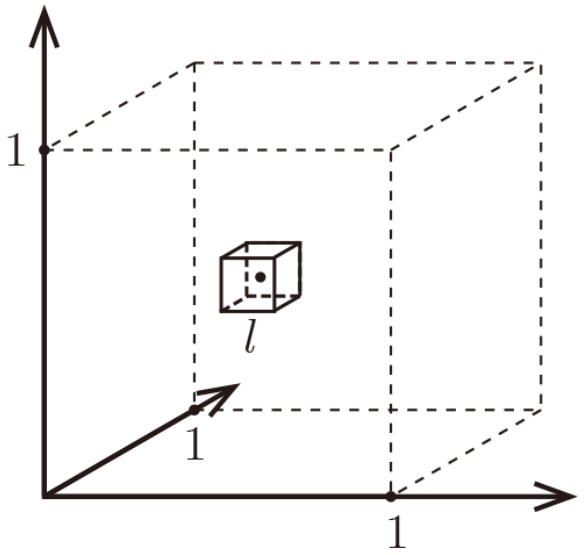
$$\frac{\text{max distance} - \text{min distance}}{\text{min distance}} \to 0 \quad \text{as } d \to \infty$$

Classical Models – kNN

- **Curse of dimensionality:**

Example:

- Consider the *d* dimensional unit square: $[0,1]^d$
- Let *m* training points be uniformly distributed
- Focus on a test point and its $k = 10$ neighbors



- Let ℓ the side of the smallest square that contains the *k* neighbors
- Then $\ell^d \approx \frac{k}{m} \Rightarrow \ell \approx \left(\frac{k}{m}\right)^{1/d}$

- For $m = 1000$

d	ℓ
2	0.1
10	0.63
100	0.955
1000	0.9954

Almost the whole space!

Classical Models – kNN

- **Curse of dimensionality:**
  - Histogram plots of the distributions of all pairwise distances between randomly distributed points within *d*-dimensional unit squares.

